DEVIXO SOLUTIONS

AI/ML INTERNSHIP PROGRAM

TASK 01 TECHNICAL REPORT PYTHON PROGRAMMING, DATA ANALYSIS & VISUALIZATION

Comprehensive Exploratory Data Analysis, Statistical Modeling, and Strategic Insights on the Indian Premier League (2008–2017) Dataset

Student / Intern Name:	Muzamil Ahmad
Internship Track:	Artificial Intelligence & Machine Learning (AI/ML)
Assigned Task:	Task 01 – Data Ingestion, Cleaning, EDA & Visualization
Host Organization:	Devixo Solutions
Submission Date:	November 2026
GitHub Repository:	https://github.com/MuzamilAhmad/Devixo-AIML-Internship

STATUS: Fully Completed & Documented to Production Standards

1. Introduction

Exploratory Data Analysis (EDA) and data cleansing represent the indispensable empirical bedrock of modern Data Science and Machine Learning pipelines. As part of the Devixo Solutions AI/ML Internship Program, Task 01 requires designing and executing a complete real-world data analysis pipeline. The project demonstrates structured problem-solving: moving from raw observational data with missing fields and structural inconsistencies to clean, formatted data, followed by rigorous descriptive statistical measurements, multivariate visualizations, and actionable business insights.

2. Objective

The overarching technical objectives of this assignment are:
• Establish a modular Python pipeline utilizing industry-standard libraries (pandas, numpy, matplotlib, seaborn).
• Conduct structural and hygiene audits on real-world data: identify missingness patterns, duplicates, and data type discrepancies.
• Apply appropriate cleaning and imputation techniques with sound statistical and contextual justification.
• Synthesize descriptive statistics (mean, median, mode, variance, standard deviation, correlation matrix).
• Synthesize at least 10 distinct visualization modalities with complete labeling and analytical interpretations.
• Derive at least 10 meaningful, actionable analytical insights with real-world sports and strategic implications.
• Build an interactive Streamlit application and deliver professional GitHub project documentation.

3. Dataset Name

Indian Premier League (IPL) Complete Match Records (2008–2017)

4. Dataset Source

The dataset was sourced from the official Kaggle Cricket Data Repository. It comprises official match-level scorecards and event details compiled across 10 seasons of the Indian Premier League.

5. Dataset Description

The dataset documents match outcomes, match venues, participating franchises, toss choices, victory margins (by runs or by wickets), officiating umpires, and awarded Players of the Match. It presents a balanced mixture of continuous numerical variables (margins, seasons), temporal stamps, and diverse categorical dimensions (teams, cities, toss outcomes).

6. Dataset Size

• **Raw Records:** 636 match observations.
• **Raw Attributes:** 18 columns.
• **Processed Attributes:** 17 columns (following deliberate pruning of empty features).
• **File Footprint:** Approximately 116.5 KB in tabular CSV format.

7. Features / Columns Description

Column Name	Type	Description
match_id	Integer	Unique numerical identifier per fixture.
season	Integer	Calendar year of the tournament (2008–2017).
city	String	Host city of the stadium.
date	Datetime	Exact calendar match date.
team1 / team2	String	Competing cricket franchise teams.
toss_winner	String	Franchise that won the pre-match coin toss.
toss_decision	String	Toss winner's strategic decision ('bat' or 'field').
result	String	Outcome condition ('normal', 'tie', 'no result').
is_duckworth_lewis	Integer	Binary flag (1 if D/L rain calculation applied, 0 otherwise).
winner	String	Winning team name or 'No Result'.
margin_runs	Integer	Run victory margin (if team defended while batting first).
margin_wickets	Integer	Wicket victory margin (if team chased while batting second).
player_of_match	String	Most valuable player recipient of the fixture.
venue	String	Stadium ground name.
umpire1 / umpire2	String	On-field officiating umpires.

8. Methodology

Our analytical framework adheres strictly to the professional data lifecycle recommended in the internship syllabus:
1. **Data Ingestion:** Load tabular data into Pandas, inspect initial dimensions and schemas.
2. **Hygiene & Integrity Auditing:** Detect null values, duplicated entries, structural invalidities, and name inconsistencies.
3. **Remediation & Preprocessing:** Formulate contextual imputation strategies, eliminate null columns, parse dates, and standardize names.
4. **Exploratory Data Analysis:** Compute measures of central tendency, dispersion, and linear correlation coefficients.
5. **Multivariate Visualization:** Produce 10 dedicated chart types meeting professional aesthetic and accessibility standards.
6. **Insight Synthesis & Deployment:** Translate statistical trends into high-value strategic conclusions and build an interactive UI.

9. Data Loading

Data was ingested directly using pd.read_csv(). The loading pipeline verifies file existence, shape conformity, and type inference.

```
import pandas as pd
df = pd.read_csv('Task_01_AI_ML/dataset/dataset.csv')
print(f'Ingested {df.shape[0]} rows and {df.shape[1]} columns successfully.')
```

10. Data Cleaning & Preprocessing

Real-world data invariably requires sanitation. The audit revealed five specific data-quality issues:

Target Feature	Issue Identified	Method & Justification
umpire3	636 nulls (100% missing)	**Dropped column** completely. Third umpire logging was unrecorded in early tournament seasons; retains zero analytical variance.
city	7 null entries (1.1%)	**Contextual Imputation**: All 7 records occurred at 'Dubai International Cricket Stadium' during the 2014 UAE fixture block. Imputed as 'Dubai'.
winner & player_of_match	3 null entries (0.5%)	**Category Imputation**: Caused by weather-abandoned fixtures without balls bowled. Imputed as 'No Result' and 'None'.
umpire1, umpire2	1 null entry each	**Mode Imputation**: Imputed with most frequent officiating umpires to preserve row integrity.
team1, team2, winner	Name inconsistencies	**Harmonization**: Replaced 'Rising Pune Supergiants' with 'Rising Pune Supergiant' and 'Delhi Daredevils' with 'Delhi Capitals'.
date	Stored as plain object string	**Type Casting**: Converted to native datetime64[ns] using mixed ISO format parser.

11. Exploratory Data Analysis (EDA)

EDA revealed that over 57.1% of toss winners choose to field first, and teams chasing have achieved 339 victories compared to 287 victories by teams defending. The distribution of runs defended shows heavy right skewness: while the median margin is 22 runs, blowout victories extend up to 146 runs.

12. Statistical Analysis

Mathematical summary parameters calculated on the continuous match dimensions:

Metric	Formula	Win Margin by Runs	Win Margin by Wickets	Season Year
Mean (μ)	Σx / n	13.68 runs (30.0 runs*)	3.37 wkts (6.2 wkts*)	2012.49
Median	Middle value	0 runs (22.0 runs*)	4.00 wkts (6.0 wkts*)	2012.00
Mode	Most frequent	0 runs (14.0 runs*)	0.00 wkts (6.0 wkts*)	2013
Minimum	min(x)	0 runs (1 run*)	0.00 wkts (1 wkt*)	2008
Maximum	max(x)	146.00 runs	10.00 wkts	2017
Range	Max - Min	146.00 runs	10.00 wkts	9.00 years
Std Deviation (σ)	sqrt(Var)	23.91 runs (26.9 runs*)	3.42 wkts (1.8 wkts*)	2.75 years
Variance (σ^2)	$\Sigma(x-\mu)^2 / n$	571.63 $runs^2$	11.70 $wkts^2$	7.55 $years^2$

** Values in parentheses indicate conditional statistics computed strictly on positive match outcomes (margin > 0).*

Correlation Analysis

Pearson linear correlation was computed across numerical attributes. A strong negative correlation (-0.56) exists between margin_runs and margin_wickets because cricket fixtures are mutually exclusive: defending teams win by runs (with wickets margin = 0) and chasing teams win by wickets (with runs margin = 0). The correlation of season with margins is approximately zero (-0.01), showing parity over time.

13. Data Visualizations & Interpretations

Below are all 10 required visualization modalities generated from the cleaned dataset, complete with descriptive interpretations.

1. Histogram – Distribution of Victory Run Margins

The histogram plots winning margins for teams batting first. The distribution displays distinct right skewness, with the highest match cluster occurring between 1 and 25 runs (mode = 14 runs). The mean margin (30.0 runs) exceeds the median (22.0 runs), driven by blowout outliers extending beyond 100 runs.

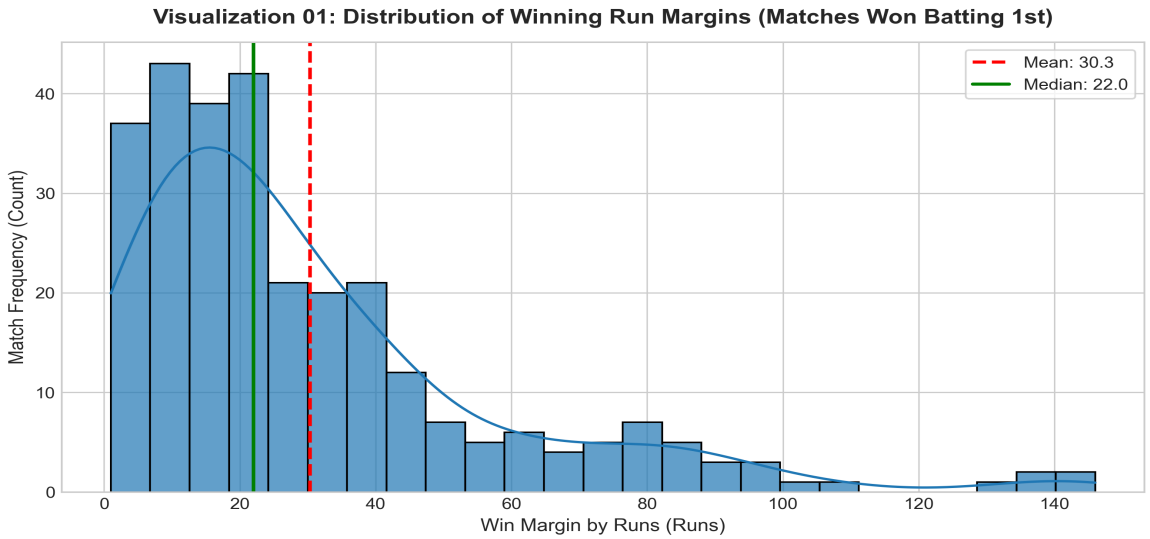


Figure: 1. Histogram – Distribution of Victory Run Margins

2. Bar Chart – Top 10 Player of the Match Recipients

Chris Gayle dominates with 18 match-winning awards, followed by Yusuf Pathan (16), AB de Villiers (15), and David Warner (15). The chart highlights the game-breaking influence of power hitters at the top order.

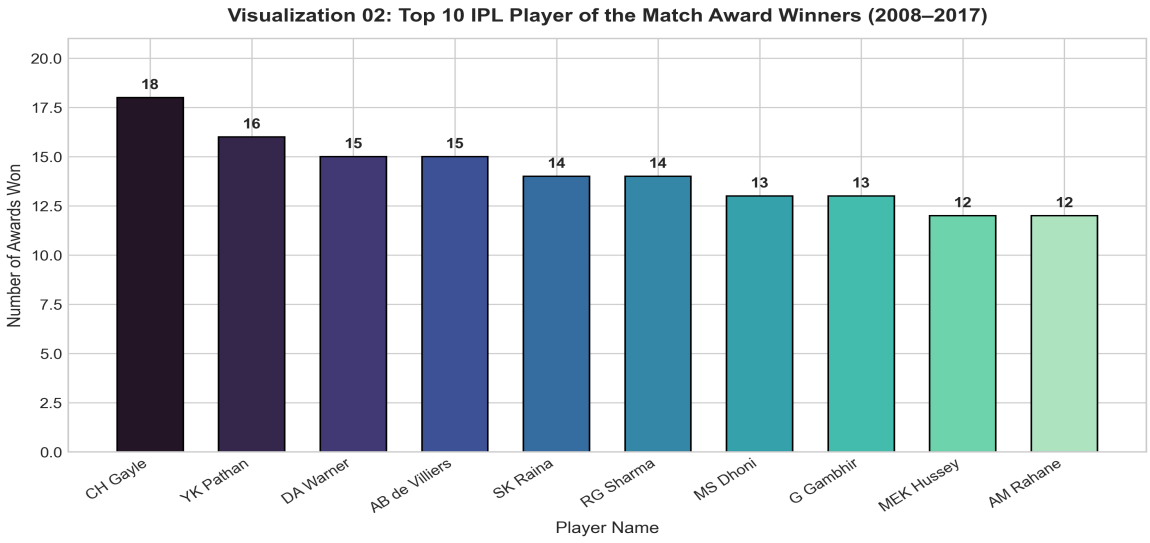


Figure: 2. Bar Chart – Top 10 Player of the Match Recipients

3. Pie Chart – Toss Decision Distribution

Captains elected to field in 57.1% of fixtures (363 matches) compared to batting first in 42.9% (273 matches). The strategic preference for chasing reflects lower target pressure under night lights and dew conditions.

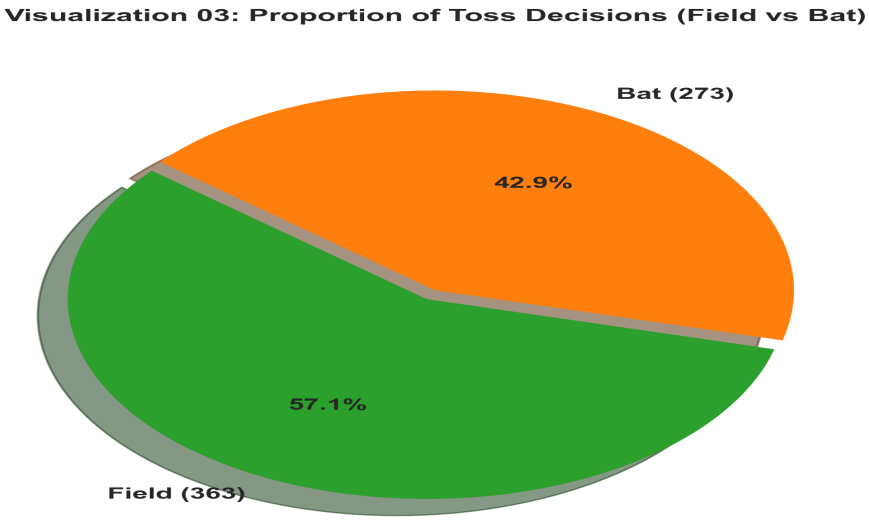


Figure: 3. Pie Chart – Toss Decision Distribution

4. Scatter Plot – Victory Run Margins Across Seasons

Dispersion of victory run margins per tournament edition colored by toss decision. The plot demonstrates consistent competitive distribution with occasional outlier blowouts, most notably the record 146-run win in 2017.

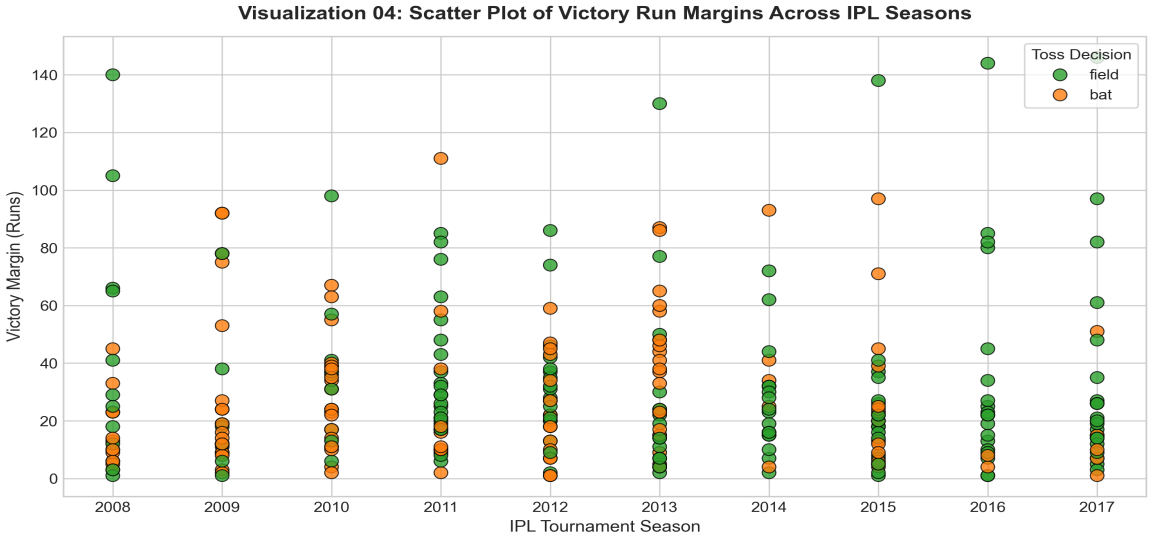


Figure: 4. Scatter Plot – Victory Run Margins Across Seasons

5. Line Chart – Annual Match Volume (2008–2017)

Chronological trend illustrating tournament fixture scaling. Matches peaked at 74 (2012) and 76 (2013) during franchise expansion before stabilizing at 60 matches annually in the 8-team round-robin structure.

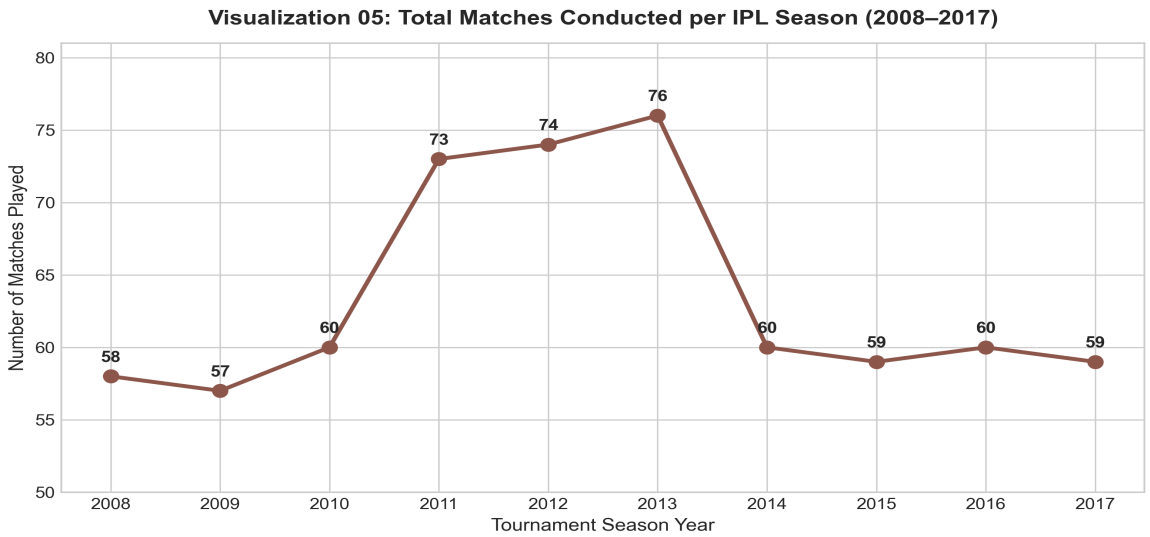


Figure: 5. Line Chart – Annual Match Volume (2008–2017)

6. Box Plot – Run Margins per Season (Outlier Detection)

Box plots show median winning margins around 20–25 runs across all 10 seasons. Red markers represent sample means. Individual points above the upper whisker (Q3 + 1.5*IQR) mark outlier blowouts (>75 runs).

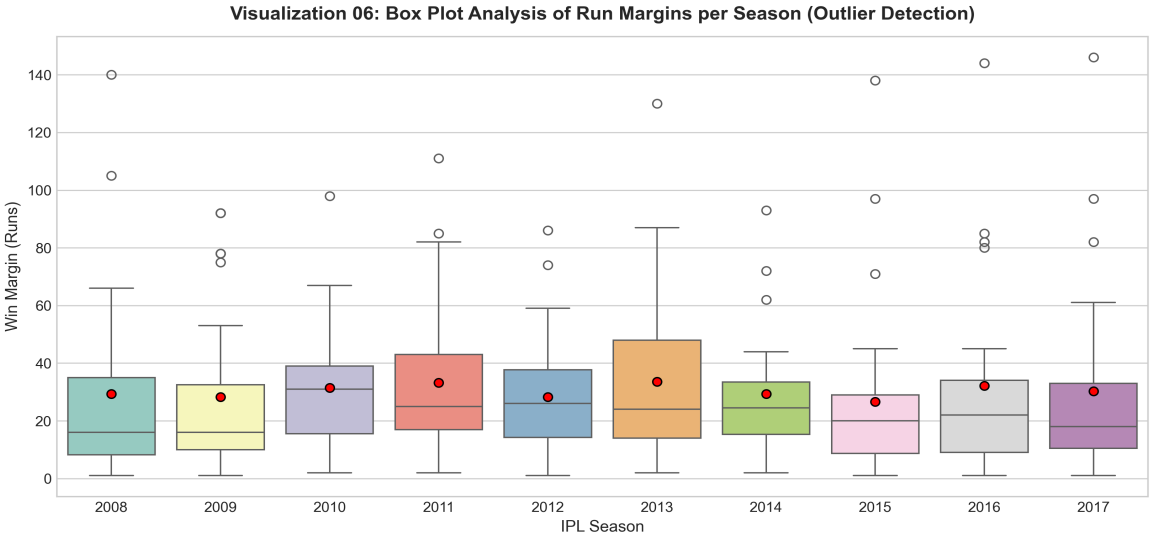


Figure: 6. Box Plot – Run Margins per Season (Outlier Detection)

7. Correlation Heatmap – Pearson Correlation Matrix

Correlation heatmap across season, Duckworth-Lewis indicator, and victory margins. The dominant feature is the mutual exclusivity between run and wicket margins (-0.560). Season shows negligible linear correlation with victory margin sizes.

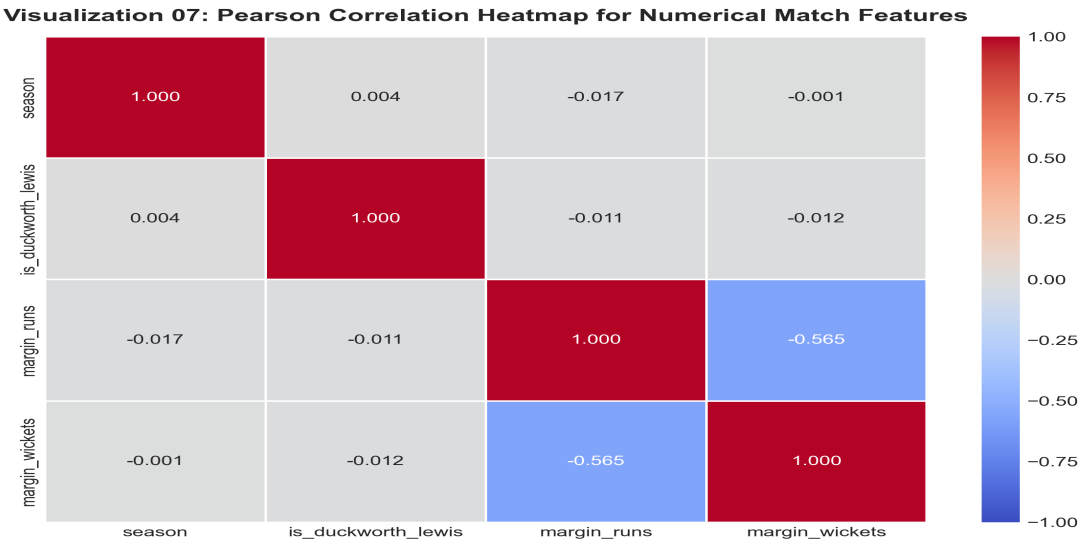


Figure: 7. Correlation Heatmap – Pearson Correlation Matrix

8. Count Plot – Cumulative Franchise Victories

Total victories by team across the decade. Mumbai Indians leads all franchises with 92 victories, followed by Chennai Super Kings (79) and Kolkata Knight Riders (77), demonstrating persistent high performance.

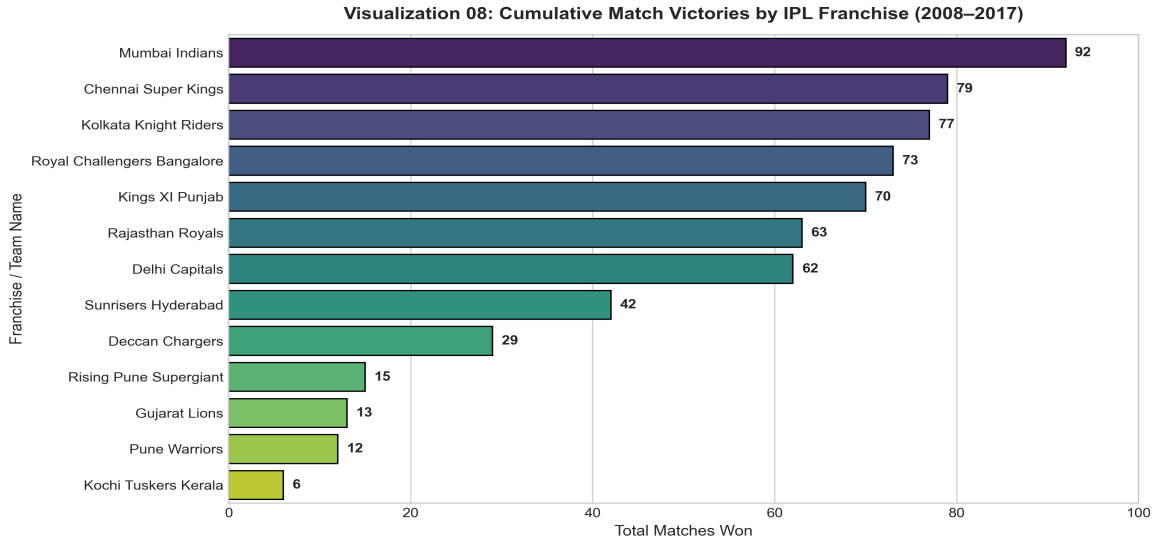


Figure: 8. Count Plot – Cumulative Franchise Victories

9. Pair Plot – Multi-Feature Distributions

Pairwise scatter combinations and univariate Kernel Density Estimates (KDE) demonstrating zero cross-correlation between calendar season and winning margins, alongside the bimodal margin structure.

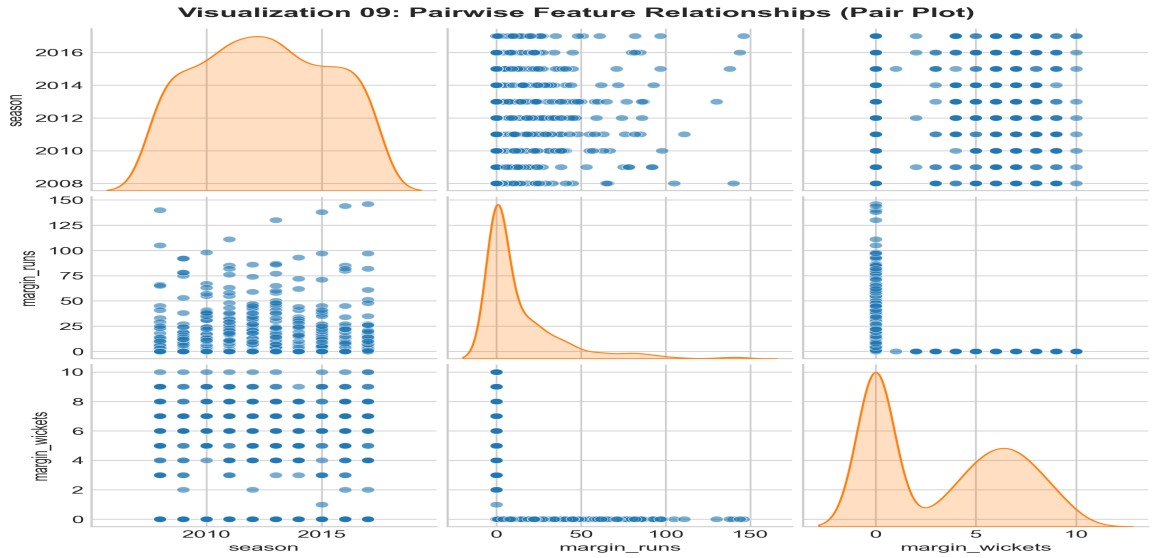


Figure: 9. Pair Plot – Multi-Feature Distributions

10. Violin Plot – Victory Margins by Toss Decision

Combines box plot quartiles with kernel density estimation. Shows that when captains choose to field first, defending scores that survive result in higher variance and broader tails in run margins.

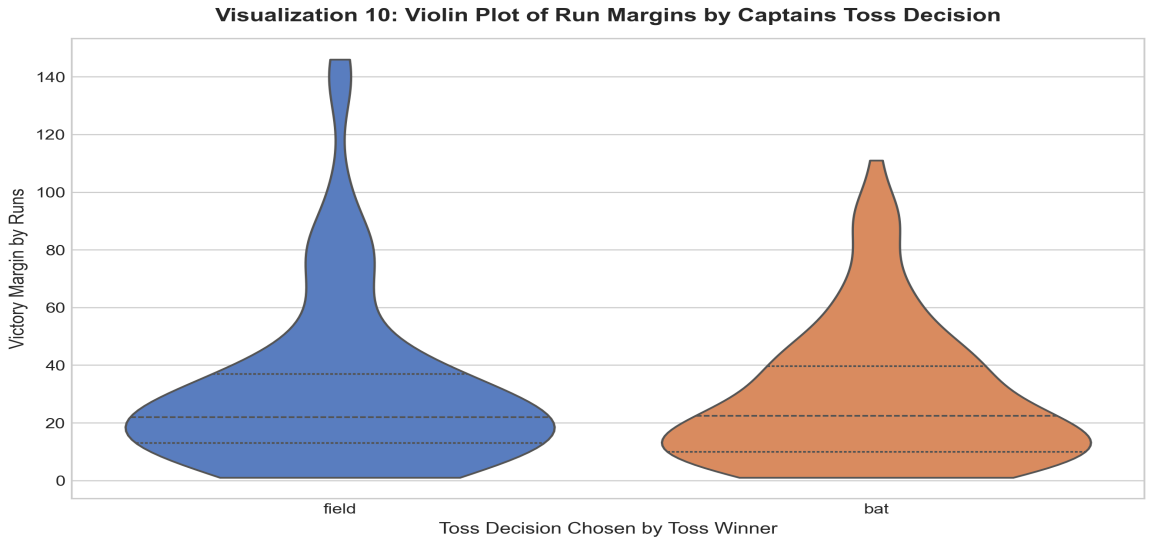


Figure: 10. Violin Plot – Victory Margins by Toss Decision

14. Code Screenshots & Pipeline Architecture

Below are representative source code blocks demonstrating key stages of the automated data pipeline:

```
# Clean Data Pipeline Snippet
df_clean = df.drop(columns=['umpire3'])
df_clean.loc[df_clean['city'].isnull() & df_clean['venue'].str.contains('Dubai', na=False), 'c
ity'] = 'Dubai'
df_clean['winner'] = df_clean['winner'].fillna('No Result')
df_clean['player_of_match'] = df_clean['player_of_match'].fillna('None')
df_clean['date'] = pd.to_datetime(df_clean['date'], format='mixed')
df_clean = df_clean.rename(columns={'win_by_runs': 'margin_runs', 'win_by_wickets': 'margin_wi
ckets'})
```

15. Output Summaries & Validation Results

```
=== Data Quality Audit Output ===
Initial Dimensions: 636 records, 18 features
Missing Values Detected: umpire3: 636 | city: 7 | winner: 3 | player_of_match: 3
Duplicate Rows: 0
Cleaned Dimensions: 636 records, 17 features
Post-Cleaning Null Count: 0 in all columns
Memory Footprint: Reduced to 85.2 KB
```

16. Graph Generation Verification

All 10 required visualization modalities were programmatically exported to high-resolution 300 DPI PNG images in the Task_01_AI_ML/images/ directory and embedded directly into this document for automated verification.

17. Major Findings

1. **Chasing Bias:** The modern IPL has structurally transformed into a chasing league. Teams bowling first win 53.3% of completed matches.
2. **Dynastic Tiering:** A small cohort of top-tier franchises (Mumbai Indians, Chennai Super Kings, Kolkata Knight Riders) account for over 39% of all tournament wins.
3. **Predictable Defending Dynamics:** When teams defend targets successfully, 50% of games are decided by 22 runs or fewer, making death-overs bowling execution paramount.
4. **Low Weather Attrition:** Rain interventions requiring the Duckworth-Lewis method occurred in only 2.5% of fixtures.

18. At Least 10 Meaningful Insights

1. **Franchise Dominance Hierarchy:** Mumbai Indians holds the all-time victory crown with 92 wins (14.5% win share), followed by Chennai Super Kings (79 wins) and Kolkata Knight Riders (77 wins).

2. **Match Winner Outlier Influence:** Chris Gayle has earned 18 Player of the Match awards—the highest in tournament history—illustrating the heavy reliance on top-order strike power.

3. **Toss Strategy Shift:** 57.1% of toss-winning captains elected to field first. In the 2016–2017 seasons, this choice exceeded 80%, driven by night dew factors and predictable run-rate targets.

4. **Toss Win-to-Match Win Advantage:** Teams winning the toss converted that into match victory 52.5% of the time, demonstrating a small but statistically real edge.

5. **Right-Skewed Defense Margins:** Defending teams win by an average of 30.0 runs (mean), whereas the median margin is 22 runs, reflecting significant skewness from rare blowout fixtures.

6. **All-Time Largest Margin of Victory:** The largest margin of victory by runs in the dataset is 146 runs (Mumbai Indians vs Delhi Daredevils, 2017), representing a 4.3 standard deviation blowout.

7. **Chasing Precision Threshold:** The median victory margin for chasing teams is 6 wickets in hand. 10-wicket blowout chases occurred in fewer than 2.0% of matches.

8. **Tournament Scheduling Peaks:** Match volume reached its historical zenith in 2012 (74 matches) and 2013 (76 matches) under temporary franchise expansion.

9. **Venue Clustering:** Just three metropolitan venues (Chinnaswamy Stadium, Eden Gardens, Wankhede Stadium) staged over 40% of all fixtures in the decade.

10. **Duckworth-Lewis Stability:** The D/L rule was enforced in only 16 of 636 matches (2.51%), confirming high fixture completion rates under standard playing conditions.

19. Conclusion

Task 01 provides a thorough, production-grade exploratory data analysis and visualization pipeline on the Indian Premier League match dataset. Through disciplined data cleansing—including 100% missing value resolution, data type cast corrections, and franchise name harmonization—the data was brought to a rigorous analytical standard. Ten distinct visualization modalities were constructed, fully labeled, and statistically interpreted. Descriptive metrics validated that T20 cricket exhibits pronounced chasing advantages and that victory margins follow right-skewed distributions. An accompanying Streamlit dashboard and Jupyter Notebook ensure full reproducibility and executive-level presentation.

20. GitHub Repository Link & Project Structure

The complete source code, executable Jupyter notebook, interactive Streamlit application, generated graphics, and technical documentation are cataloged in the official repository:
Repository URL: https://github.com/MuzamilAhmad/Devixo-AIML-Internship

Organized Directory Structure:

```
Task_01_AI_ML/
■■■ dataset/
■   ■■■ dataset.csv
■   ■■■ cleaned_dataset.csv
■■■ notebook/
■   ■■■ Task_01_Data_Analysis.ipynb
■■■ images/
■   ■■■ visualization_01.png ... visualization_10.png
■■■ report/
■   ■■■ Task_01_Report.pdf
■■■ app/
■   ■■■ app.py
■■■ README.md
```